CLASS DIAGRAMS

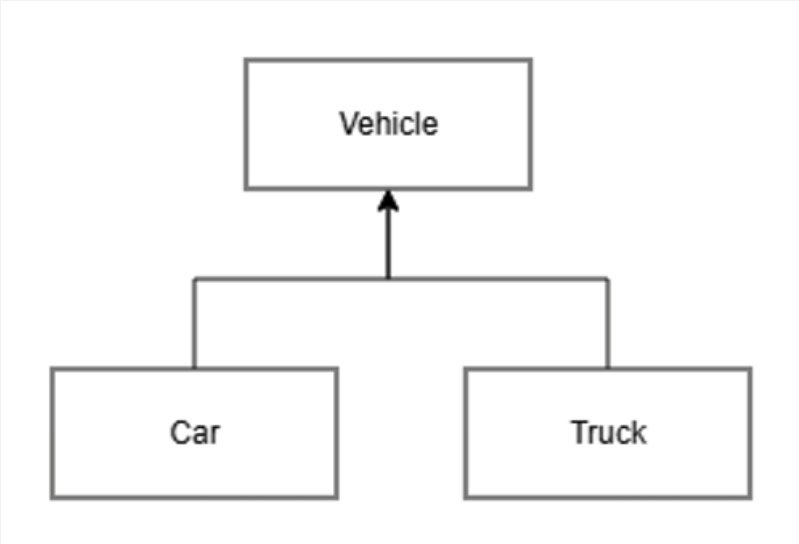
UML Style Class Diagram Basics

LEARNING GOALS

- By the end of today's class, I will be able to:
  - Read and analyze simple Class Diagrams
  - Use a tool to create simple Class Diagrams
  - Describe how inheritance and dependency relationships are indicated on a Class Diagram
  - Represent private, public and protected attributes (variables) and behaviours (methods) are represented on a Class Diagram
  - Distinguish between Conceptual, Specification and Implementation Diagrams

WHAT IS A CLASS DIAGRAM?

- A Class Diagram is a way to **represent the relationship** between classes within a program
- A Class Diagram is a UML standard (Unified Modeling Language – an agreed-upon set of standards to ensure that professionals all around the world use the same techniques, and can thus work together)
- A Class Diagram may also include detail about each class, though only the class name is *required*

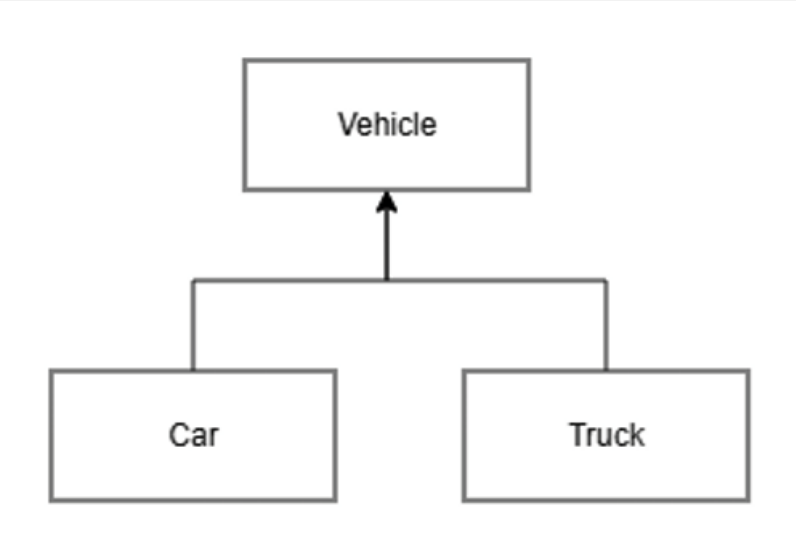


VARIOUS LEVELS OF DETAIL

- Not every Class Diagram is the same
- Class diagrams come in different levels of detail
  - Concept (least detailed) → Specification → Implementation (fully detailed)
  - With or Without **Signature** (methods and variables have types indicated)

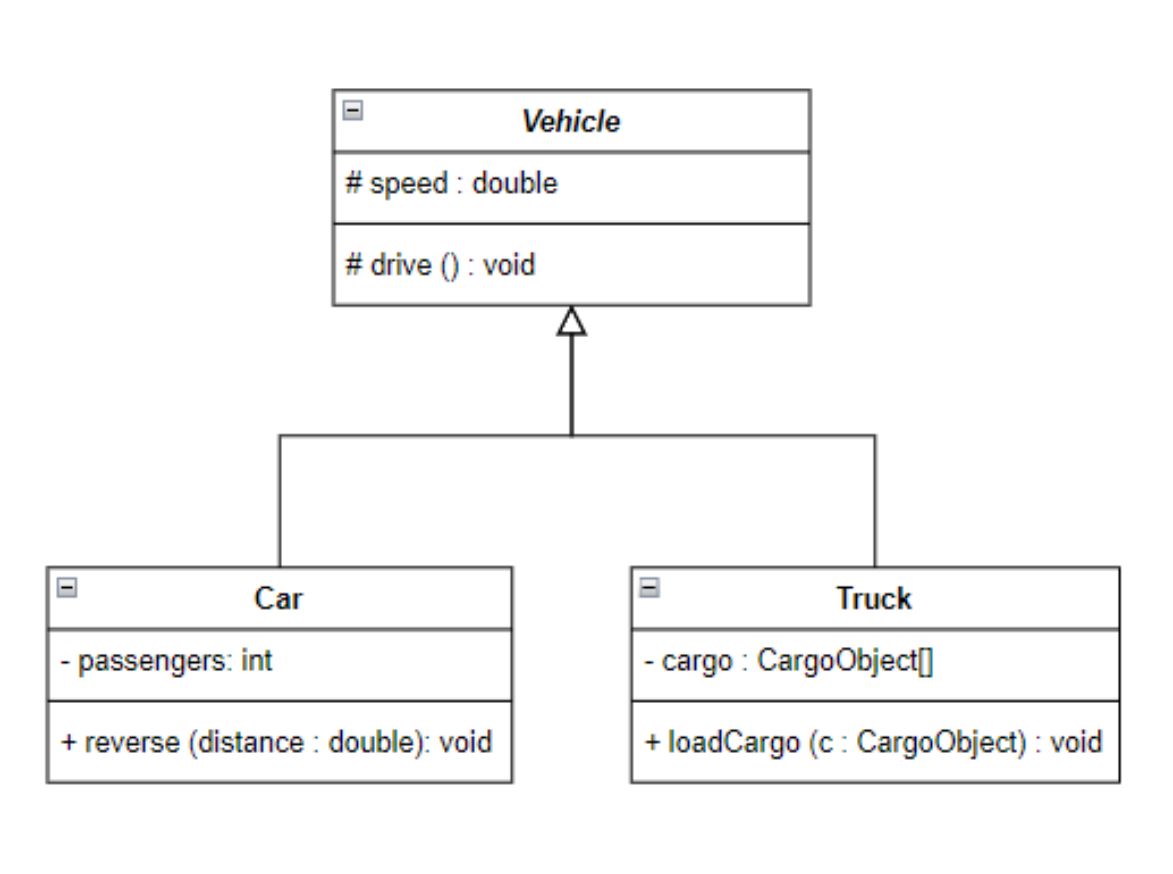
CONCEPT LEVEL EXAMPLE

- Solid line with arrowhead represents **inheritance**
- <u>The only required information on a class diagram is the class name and the relationship arrows</u>
- When only class names and relationships are described, we refer to this as a **Concept Level** class diagram



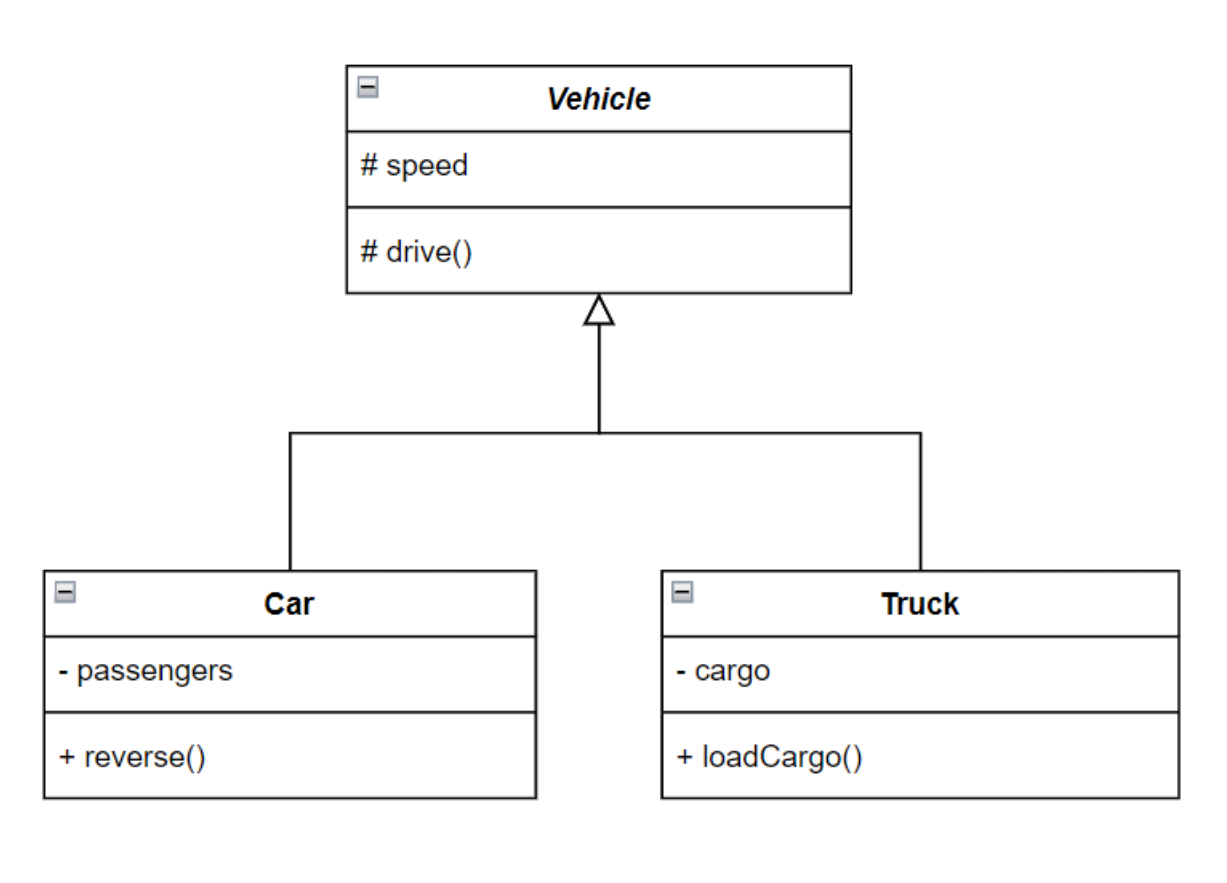
SPECIFICATION LEVEL EXAMPLE

- This Class Diagram is a Specification Level diagram, and includes **critical (important)** methods and variables, but not necessarily all methods and variables
- Each Variable and method has it's access level indicated:
  - **+** means public
  - **–** means private
  - **#** means protected



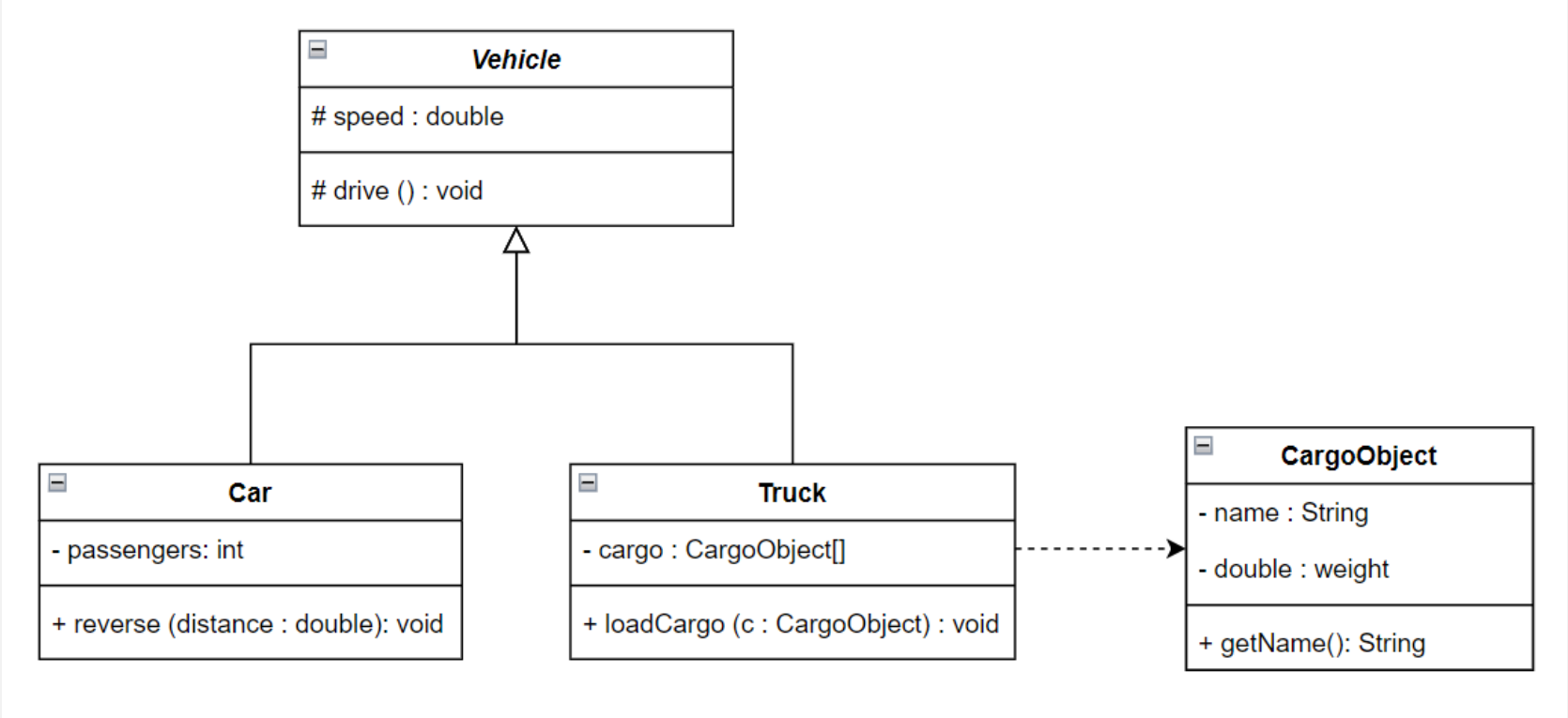
NO SIGNATURE

- This is also a **Specification Level** Class Diagram
- However, it was created **without signatures** – in other words, critical methods and variables, as well as their access levels, are listed the same as before, **but their types, parameters and return types are left out**



INHERITS & DEPENDENCY LINES

- A **solid line** with an arrowhead represents **inheritance**
  - For example, Truck <u>inherits</u> from Vehicle
- A **dashed line** with an arrowhead represents a **dependency** relationship
  - For example, Truck uses and therefore <u>depends on</u> CargoObject

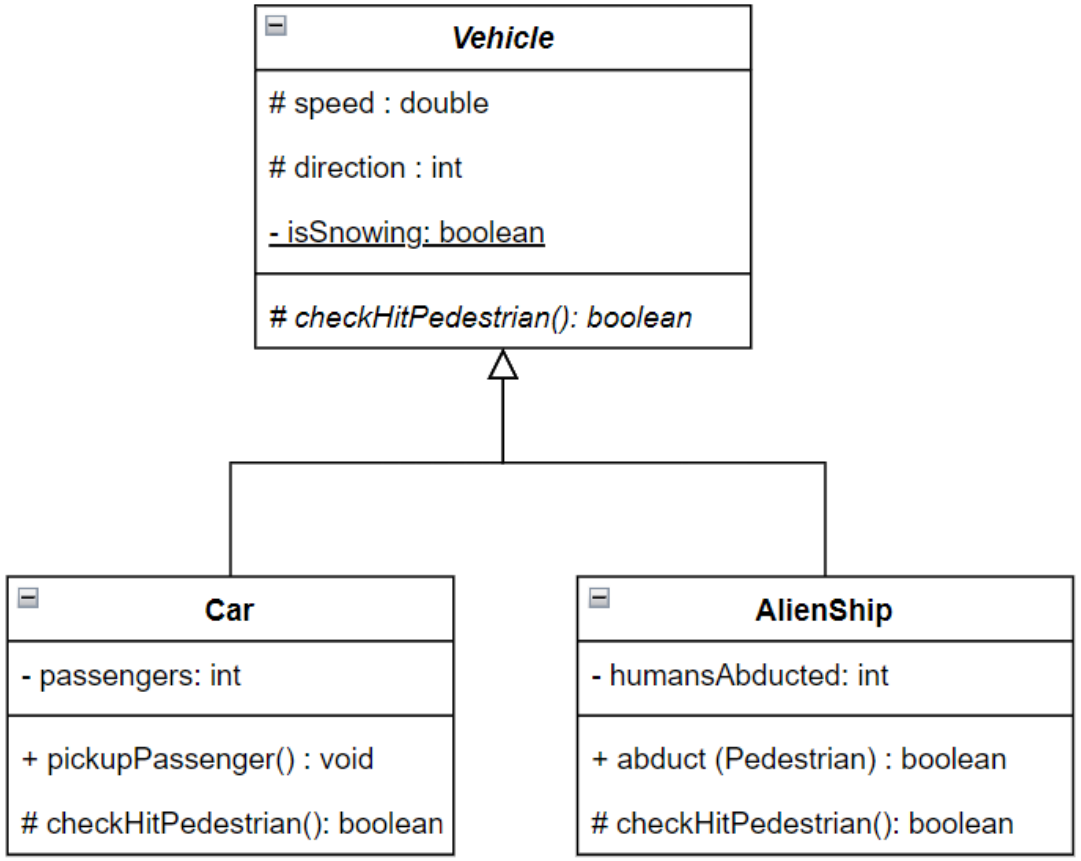


IMPLEMENTATION LEVEL DIAGRAM

- An **Implementation Level Class Diagram** is very much like a Specification Level Class Diagram except that it **lists every method and variable** and not only the critical/important ones.

ABSTRACT AND STATIC

- Abstract super-classes can either be written in ***italics*** or marked with the text **`{abstract}`** next to the class name. (Both are accepted UML Standard as not all environments can support italic font).
- Abstract methods are indicated in ***italics*** and should also be shown in the subclasses in regular text
- Static methods and variables should be **underlined**

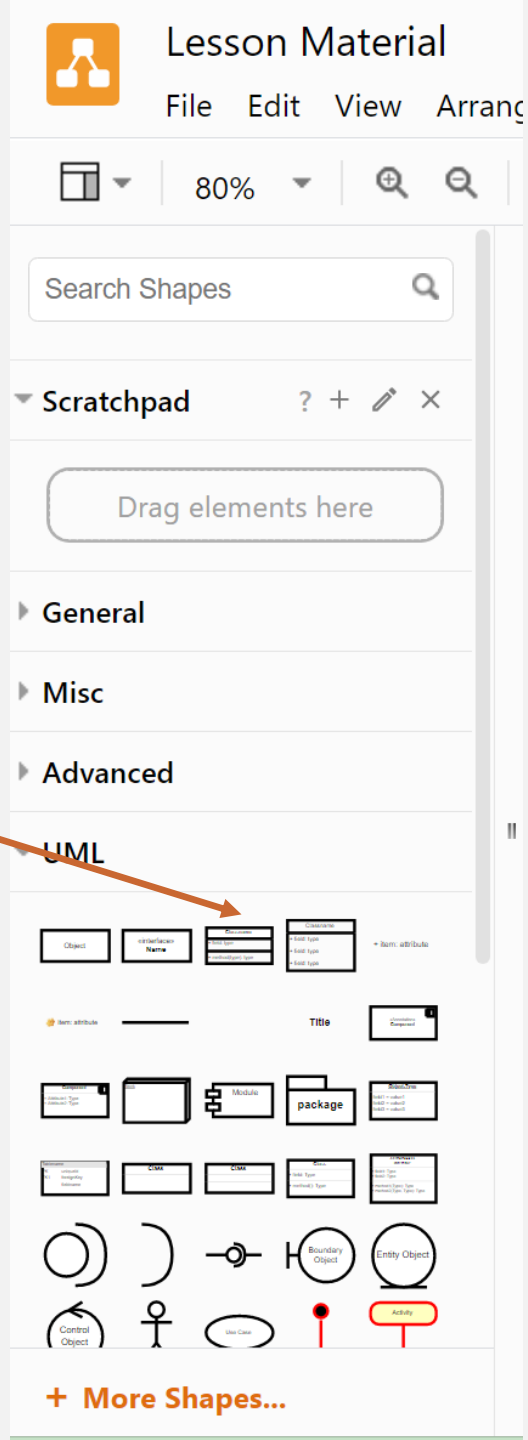


SUMMARY

- Three levels of Class Diagram
  - Concept → Only Class Names, Relationships
  - Specification → Includes critical/important Traits (variables) and Behaviors (methods)
  - Implementation → Same as Specification, but with all traits and behaviors listed, so that you can implement (code) the class and not miss any details
- Signatures
  - Class diagrams may or may not include signatures – variables types, method parameter and return types

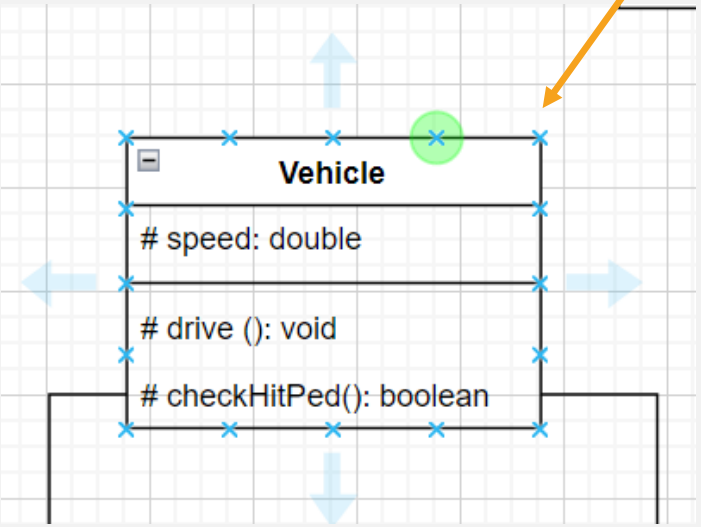
MAKING CLASS DIAGRAMS

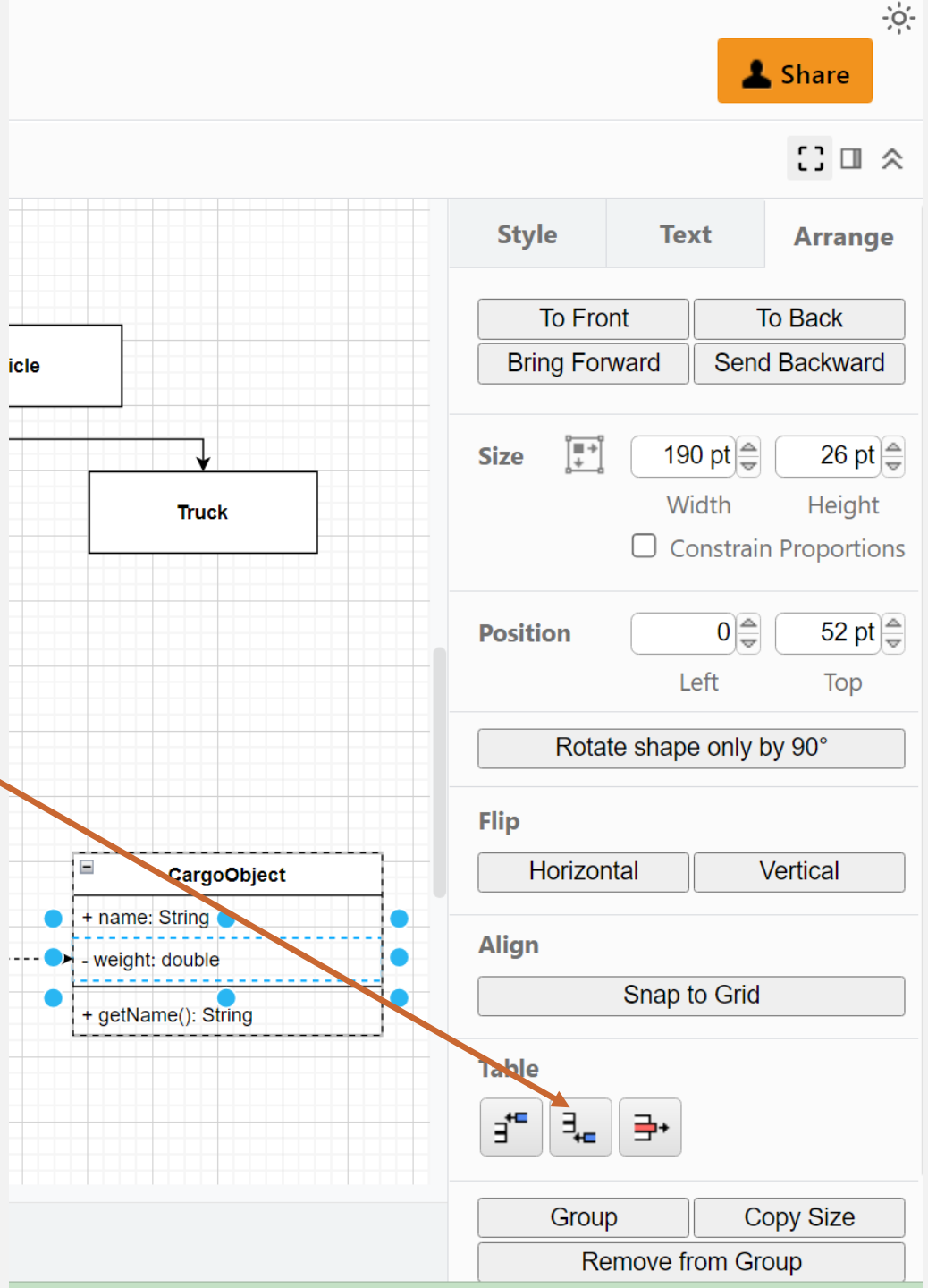
- We will use the same website that we used for Flow Charts – app.diagrams.net
- We will use the Class tool from the UML tab, as pictured



MAKING CLASS DIAGRAMS

- To add connections, select the class box and then mouse over a class box and choose one of the blue anchor points, then draw a line to another class box
- To add additional lines, use the Insert Row button in the Arrange Tab, as pictured





TASK

- Your task is to create a **Specification Level class diagram** for your (already completed) Vehicle Simulation Project
- You should include the **primary** (most important) classes, which will include Pedestrian and Vehicle, as well as their subclasses. It should also include the **World** and any other classes that you deem to be important. For clarity, this means that smaller helper classes can be left out.
- You should decide which behaviours and traits (aka methods and variables) are important – you are not required to provide every single one.
- (Optional) You can improve your diagram by adding Actor and SuperSmoothMover as top-level classes, which will connect your primary classes.